

Dungeon Game Logic

The key to **Dungeon Game** is to think **backward**, not forward.

The biggest trap is trying to calculate the maximum health you can collect along a path. That doesn't work because the knight can die **before** reaching a healing room.

1. What are we actually trying to find?

Suppose we are standing at some cell (i, j) .

We need to know:

What is the minimum health I must have BEFORE entering (i, j) so that I can safely reach the princess from there?

Let's call this:

```
\[
dp[i][j] = \text{minimum health required before entering cell }(i,j)
\]
```

This definition is the entire problem.

2. Why do we work backward?

Consider the last cell:

text

$[-5]$

The knight enters this cell and loses 5 health.

He needs at least 1 health **after** taking the damage.

Therefore:

text

required health = 1 + 5 = 6

So:

```
\[
dp[m-1][n-1] = \max(1, 1-dungeon[m-1][n-1])
\]
```

For -5:

```
\[
dp = \max(1, 1-(-5)) = 6
\]
```

For +10:

text

Before entering: 1 health
After gaining 10: 11 health

So only 1 health is necessary:

```
\[
dp = \max(1, 1-10)=1
\]
```

This is why backward DP is natural.

3. Recurrence relation

Suppose we are at:

text

(i, j)

From here the knight has only two choices:

text

$$\begin{array}{ccc} & (i, j) & \\ & / \quad \backslash & \\ \downarrow & & \rightarrow \\ (i+1, j) & & (i, j+1) \end{array}$$

We already know how much health is required from those next cells.

So:

```

text
right = dp[i][j+1]
down  = dp[i+1][j]

```

Obviously, we should choose the path requiring **less health**:

```

\[
next = \min(dp[i+1][j], dp[i][j+1])
\]

```

Now consider the current dungeon value:

```

text
dungeon[i][j]

```

Suppose:

```

text
dungeon[i][j] = -3
next = 5

```

We need 5 health after entering the current room.

Since this room takes 3 health:

```

text
required before current room = 5 + 3 = 8

```

Mathematically:

```

\[
required = dp[next] - dungeon[i][j]
\]

```

So:

```

\[
dp[i][j]
=
\max(1, \min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j])
\]

```

★ Main recurrence

```

\[
\boxed{
dp[i][j]
=
\max(1, \min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j])
}
\]

```

The $\max(1, \dots)$ is extremely important because the knight must **never have health ≤ 0** .

4. Let's understand with the actual example

```

text
-2  -3  3
-5 -10  1
10  30 -5

```

Start from bottom-right.

Cell (2,2) = -5

Need:

```

text
1 - (-5) = 6

```

So:

```

text
6

```

Cell (2,1) = 30

The next cell requires 6.

We gain 30 here:

text

$$6 - 30 = -24$$

But health can't be below 1.

Therefore:

text

$$dp[2][1] = 1$$

Cell (2,0) = 10

Next cell requires 1.

text

$$1 - 10 = -9$$

So:

text

$$dp[2][0] = 1$$

Bottom row becomes:

text

$$1 \quad 1 \quad 6$$

Now move to (1,2) = 1.

The only next cell is (2,2) requiring 6.

We gain 1:

text

$$6 - 1 = 5$$

Therefore:

text

$$dp[1][2] = 5$$

At (1,1) = -10:

We can go:

text

down -> 1
right -> 5

Choose the cheaper path:

text

$$\min(1, 5) = 1$$

Current room costs 10:

text

$$1 - (-10) = 11$$

Therefore:

text

$$dp[1][1] = 11$$

At (1,0) = -5:

text

down = 1
right = 11

$$\min = 1$$

$$\begin{aligned} \text{required} &= 1 - (-5) \\ &= 6 \end{aligned}$$

So:

text

```
dp[1][0] = 6
```

Continue upward.

Eventually:

text

```
7 5 3
6 11 5
1 1 6
```

Therefore:

text

```
answer = dp[0][0] = 7
```

5. The intuition you should remember

Don't think:

"How much health can I collect?"

Think:

"How much health must I have right now to survive the rest of the journey?"

That's the DP state.

At every cell:

1. Look at the required health of the two possible next cells.
2. Choose the smaller one.
3. Account for the current room's damage/healing.
4. Make sure the answer is at least 1.

6. Memoization / Top-Down code

Since you're learning DP patterns, let's first write the recursive version.

C++

```
class Solution {
public:
    int m, n;
    vector<vector<int>> dp;

    int solve(int i, int j, vector<vector<int>>& dungeon) {
        // Outside the grid = impossible path
        if (i >= m || j >= n)
            return 1e9;

        // Princess cell
        if (i == m - 1 && j == n - 1)
            return max(1, 1 - dungeon[i][j]);

        if (dp[i][j] != -1)
            return dp[i][j];

        int right = solve(i, j + 1, dungeon);
        int down = solve(i + 1, j, dungeon);

        int nextHealth = min(right, down);

        return dp[i][j] = max(1, nextHealth - dungeon[i][j]);
    }

    int calculateMinimumHP(vector<vector<int>>& dungeon) {
        m = dungeon.size();
        n = dungeon[0].size();

        dp.assign(m, vector<int>(n, -1));

        return solve(0, 0, dungeon);
    }
};
```

7. Why `1e9` for outside the grid?

Consider:

```
text
    current
      ↓
    right  down
```

At the right edge, there is no right cell.

We don't want the algorithm to choose that nonexistent path.

So:

```
C++
if (i >= m || j >= n)
    return 1e9;
```

means:

```
"This path is impossible, so make it extremely expensive."
```

Then:

```
C++
min(right, down)
```

will automatically choose the valid path.

8. Why `max(1, ...)`?

This is another very important part.

Suppose:

```
text
current cell = +10
nextHealth = 5
```

Then:

```
text
required = 5 - 10
          = -5
```

Mathematically we would get `-5`.

But the knight cannot start with `-5` health.

Even more importantly, the knight must always have **at least 1 HP**.

Therefore:

```
C++
max(1, nextHealth - dungeon[i][j])
```

So:

```
text
max(1, -5) = 1
```

9. Tabulation / Bottom-Up

Now convert the recurrence into iterative DP.

We start from the bottom-right and move toward the top-left.

```
C++
class Solution {
public:
    int calculateMinimumHP(vector<vector<int>>& dungeon) {
        int m = dungeon.size();
        int n = dungeon[0].size();

        vector<vector<int>> dp(m, vector<int>(n));

        // Princess cell
        dp[m - 1][n - 1] = max(1, 1 - dungeon[m - 1][n - 1]);
```

```

// Last row
for (int j = n - 2; j >= 0; j--) {
    dp[m - 1][j] = max(1, dp[m - 1][j + 1] - dungeon[m - 1][j]);
}

// Last column
for (int i = m - 2; i >= 0; i--) {
    dp[i][n - 1] = max(1, dp[i + 1][n - 1] - dungeon[i][n - 1]);
}

// Remaining cells
for (int i = m - 2; i >= 0; i--) {
    for (int j = n - 2; j >= 0; j--) {
        int nextHealth = min(dp[i + 1][j], dp[i][j + 1]);

        dp[i][j] = max(1, nextHealth - dungeon[i][j]);
    }
}

return dp[0][0];
}
};

```

10. Can we do it without an extra DP array?

Yes.

And this is actually a nice optimization for this problem because the **dungeon matrix itself can be converted into the DP table**.

You update every cell with the minimum health required from that cell.

```

C++
class Solution {
public:
    int calculateMinimumHP(vector<vector<int>>& dungeon) {
        int m = dungeon.size();
        int n = dungeon[0].size();

        dungeon[m - 1][n - 1] = max(1, 1 - dungeon[m - 1][n - 1]);

        // Last row
        for (int j = n - 2; j >= 0; j--) {
            dungeon[m - 1][j] = max(1, dungeon[m - 1][j + 1] - dungeon[m - 1][j]);
        }

        // Last column
        for (int i = m - 2; i >= 0; i--) {
            dungeon[i][n - 1] = max(1, dungeon[i + 1][n - 1] - dungeon[i][n - 1]);
        }

        // Remaining cells
        for (int i = m - 2; i >= 0; i--) {
            for (int j = n - 2; j >= 0; j--) {
                int nextHealth = min(dungeon[i + 1][j], dungeon[i][j + 1]);

                dungeon[i][j] = max(1, nextHealth - dungeon[i][j]);
            }
        }

        return dungeon[0][0];
    }
};

```

This gives:

text

Original dungeon:

```

-2  -3  3
-5 -10  1
10  30 -5

```

After DP:

```
text
7  5  3
6 11  5
1  1  6
```

And:

```
C++
return dungeon[0][0];
```

gives:

```
text
7
```

Complexity

With the in-place approach:

```
\[
Time = O(m \times n)
\]
```

and

```
\[
Extra\ Space = O(1)
\]
```

because we're modifying the existing matrix.

The DP pattern to memorize

For problems like this, don't memorize the code. Memorize these four questions:

1. What does my state mean?

```
text
dp[i][j] = minimum health required before entering (i,j)
```

2. Where do I start?

At the destination, because the answer depends on surviving **to the end**.

3. Which next state do I choose?

```
C++
min(right, down)
```

because we want the path requiring the least initial health.

4. How does the current cell affect the answer?

```
C++
required = nextHealth - dungeon[i][j];
```

and:

```
C++
required = max(1, required);
```

So the entire problem reduces to:

```
C++
nextHealth = min(right, down);

currentHealth = max(1, nextHealth - dungeon[i][j]);
```

That is the core intuition behind **174. Dungeon Game**.

Absolutely. Let's do a **complete dry run** of the first example using the **in-place DP code**, so you can see exactly how every cell changes.

Input

```
C++
dungeon = {
    {-2, -3, 3},
    {-5, -10, 1},
    {10, 30, -5}
};
```

Expected answer:

text

7

The important thing: we start from the bottom-right and move toward $(0,0)$.

1. Initial matrix

text

	j=0	j=1	j=2
i=0	-2	-3	3
i=1	-5	-10	1
i=2	10	30	-5

Initially these values represent **damage/health changes**.

After processing, they will represent:

minimum health required before entering that cell.

This is why the matrix changes meaning during the algorithm.

2. Start at bottom-right $(2,2)$

Value:

text

dungeon[2][2] = -5

This is the princess cell.

The formula is:

C++

dungeon[2][2] = max(1, 1 - dungeon[2][2]);

Substitute:

text

```
= max(1, 1 - (-5))
= max(1, 6)
= 6
```

So:

text

-2	-3	3
-5	-10	1
10	30	6

What does 6 mean?

It means:

The knight needs at least 6 HP before entering $(2,2)$.

Because:

text

```
6 HP
↓
enter -5 room
↓
1 HP
```

He survives.

3. Process last row

Last row is:

text

10 30 6

We move from right to left.

Cell (2,1)

Original value:

```
text
dungeon[2][1] = 30
```

Next cell:

```
text
dungeon[2][2] = 6
```

Meaning:

After entering (2,1), we need to have 6 HP remaining.

Current cell gives us 30 HP.

Formula:

```
C++
dungeon[2][1] = max(1, dungeon[2][2] - dungeon[2][1]);
```

Substitute:

```
text
= max(1, 6 - 30)
= max(1, -24)
= 1
```

So:

```
text
-2  -3  3
-5 -10  1
10  1  6
```

Why only 1 HP?

Suppose we enter this room with just:

```
text
1 HP
```

Then the room gives us:

```
text
+30
```

So:

```
text
1 + 30 = 31 HP
```

We have much more than the required 6.

Therefore minimum is simply:

```
text
1 HP
```

4. Cell (2,0)

Original:

```
text
dungeon[2][0] = 10
```

Next cell requires:

```
text
dungeon[2][1] = 1
```

Formula:

```
text
max(1, 1 - 10)
```

Therefore:

```
text
max(1, -9) = 1
```

Matrix becomes:

```
text
-2  -3  3
-5 -10  1
 1   1  6
```

So the last row is now:

```
text
1   1  6
```

Meaning:

```
text
Before entering (2,0): need 1 HP
Before entering (2,1): need 1 HP
Before entering (2,2): need 6 HP
```

5. Process last column

Now we move upward.

Cell (1,2):

```
text
dungeon[1][2] = 1
```

The next cell is:

```
text
dungeon[2][2] = 6
```

We need 6 HP after leaving the current cell.

Current cell gives us +1.

Therefore:

```
text
required = 6 - 1
          = 5
```

So:

```
text
dungeon[1][2] = max(1, 5)
               = 5
```

Matrix:

```
text
-2  -3  3
-5 -10  5
 1   1  6
```

Meaning

To go:

```
text
(1,2) → (2,2)
```

we need:

```
text
5 HP
```

Because:

text

5
↓
+1
↓
6
↓
-5
↓
1

Perfect.

6. Cell (0,2)

Original:

text

dungeon[0][2] = 3

Next cell:

text

dungeon[1][2] = 5

Formula:

text

required = 5 - 3
 = 2

So:

text

dungeon[0][2] = max(1, 2)
 = 2

Matrix:

text

-2	-3	2
-5	-10	5
1	1	6

Meaning:

Need only 2 HP before entering (0,2) .

Because:

text

2
↓
+3
↓
5
↓
+1
↓
6
↓
-5
↓
1

7. Now process the middle cells

Now we start:

C++

```
for (int i = m - 2; i >= 0; i--)
```

and

```
C++
for (int j = n - 2; j >= 0; j--)
```

So:

```
text
i = 1
j = 1
```

We process (1,1) .

8. Cell (1,1) = -10

Current matrix:

```
text
-2  -3  2
-5  -10 5
1   1  6
```

Current cell:

```
text
dungeon[1][1] = -10
```

There are two choices.

Right

```
text
(1,1) → (1,2)
```

Required health:

```
text
dungeon[1][2] = 5
```

Down

```
text
(1,1) → (2,1)
```

Required health:

```
text
dungeon[2][1] = 1
```

So:

```
C++
nextHealth = min(5, 1);
```

Therefore:

```
text
nextHealth = 1
```

This is the crucial decision.

The algorithm says:

From (1,1) , going DOWN requires only 1 HP, while going RIGHT requires 5 HP.

Therefore choose DOWN.

Now account for current room -10 .

```
text
required = nextHealth - dungeon[1][1]
          = 1 - (-10)
          = 11
```

So:

```
text
dungeon[1][1] = 11
```

Matrix:

text

-2	-3	2
-5	11	5
1	1	6

9. Why is it 11?

This is worth understanding.

Path from (1,1) if we go DOWN:

text

(1,1) = -10
(2,1) = +30
(2,2) = -5

Suppose we start with 11 HP.

text

Start
11 HP
↓
enter -10
↓
1 HP
↓
enter +30
↓
31 HP
↓
enter -5
↓
26 HP

We survive easily.

Could we start with 10?

text

10
↓
-10
↓
0

The knight dies immediately.

So:

text

10 ✖
11 ✔

Therefore:

text

dp[1][1] = 11

10. Cell (1,0) = -5

Current matrix:

text

-2	-3	2
-5	11	5
1	1	6

At (1,0) we have two choices.

Right

text

dp[1][1] = 11

Down

text

```
dp[2][0] = 1
```

Choose minimum:

text

```
nextHealth = min(11, 1)
             = 1
```

Current cell is -5.

Therefore:

text

```
required = 1 - (-5)
          = 6
```

So:

text

```
dungeon[1][0] = 6
```

Matrix:

text

-2	-3	2
6	11	5
1	1	6

11. Cell (0,1) = -3

Now:

text

-2	-3	2
6	11	5
1	1	6

At (0,1):

Down

text

```
dp[1][1] = 11
```

Right

text

```
dp[0][2] = 2
```

Choose:

text

```
nextHealth = min(11, 2)
             = 2
```

Current cell:

text

```
-3
```

Therefore:

text

```
required = 2 - (-3)
          = 5
```

So:

text

```
dungeon[0][1] = 5
```

Matrix:

text

```
-2  5  2
 6 11  5
 1  1  6
```

12. Finally (0,0)

This is the starting cell.

Current value:

text

```
dungeon[0][0] = -2
```

Two choices:

Right

text

```
dp[0][1] = 5
```

Down

text

```
dp[1][0] = 6
```

Choose the smaller:

text

```
nextHealth = min(5, 6)
             = 5
```

Current cell has:

text

```
-2
```

Therefore:

text

```
required = 5 - (-2)
           = 7
```

So:

text

```
dungeon[0][0] = 7
```

Final DP matrix:

text

7	5	2
6	11	5
1	1	6

Therefore:

C++

```
return dungeon[0][0];
```

returns:

text

```
7
```

13. Now let's trace the actual optimal path

The DP tells us which direction requires less health.

Starting at (0,0):

text

$dp[0][0] = 7$

Choices:

text

right = 5
down = 6

Choose **RIGHT**.

At (0,1):

text

right = 2
down = 11

Choose **RIGHT**.

At (0,2):

Only down:

text

down = 5

Choose **DOWN**.

At (1,2):

Only down:

text

down = 6

Choose **DOWN**.

Reach princess:

text

(2,2)

So path is:

text

RIGHT → RIGHT → DOWN → DOWN

Exactly as the problem says.

14. Actual health simulation

This is another important way to verify the answer.

Start with:

text

7 HP

(0,0) = -2

text

7 - 2 = 5

Health:

text

5

(0,1) = -3

text

5 - 3 = 2

Health:

text

2

`(0,2) = +3`

text

`2 + 3 = 5`

Health:

text

`5`

`(1,2) = +1`

text

`5 + 1 = 6`

Health:

text

`6`

`(2,2) = -5`

text

`6 - 5 = 1`

Health:

text

`1`

The knight survives.

Therefore:

text

`minimum initial health = 7`

15. The entire algorithm in one picture

text

Original dungeon

```
-2  -3   3
-5 -10   1
10  30  -5
```

↓

Start from bottom-right

```
-2  -3   2
-5 -10   5
1   1   6
```

↓

Process remaining cells

```
-2   5   2
6  11   5
1   1   6
```

↓

Process starting cell

```
7   5   2
6  11   5
1   1   6
```

↓

`answer = 7`

The most important thing to understand

The transformed matrix is not storing health after entering the cell.

It stores:

minimum health required BEFORE entering that cell.

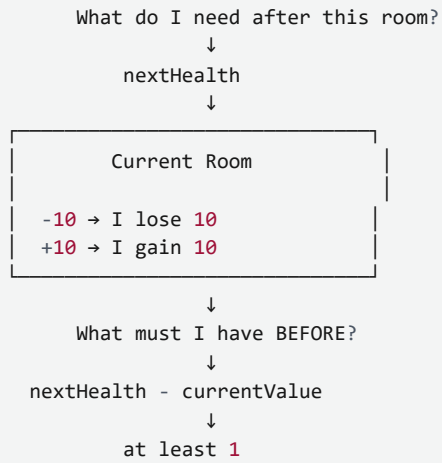
That's why the recurrence is:

C++

```
int nextHealth = min(dungeon[i + 1][j], dungeon[i][j + 1]);  
  
dungeon[i][j] = max(1, nextHealth - dungeon[i][j]);
```

Think of it as:

text



That backward perspective is the key DP insight in **Dungeon Game**.